Compilado de Notas do Caderno

iTest Learning: O Ensino do Planejamento de Testes de Software

Tópicos Integrados: Sistemas Legados com IA | Engenharia Reversa & BDD | Qualidade de Testes com JaCoCo e PIT

1. Fórum de Discussão: Refatoração e Reconstrução de Sistemas Legados com IA

1. Extração de Requisitos de Sistemas Legados com Inteligência Artificial

A. Descrição da Abordagem e Suas Etapas

A abordagem baseia-se na **Engenharia Reversa Auxiliada por Modelos de Linguagem de Grande Escala (LLMs) e Padrões de Prompt**. O objetivo é recuperar a documentação perdida ou desatualizada de sistemas legados a partir da análise estática do código-fonte (ex.: COBOL, Java legado ou C++), convertendo regras implícitas em requisitos funcionais e especificações estruturadas.

• **1. Pré-processamento e Análise Estrutural:** Parsing do código-fonte legado e geração de Árvores de Sintaxe Abstrata (AST) ou grafos de fluxo de dados, dividindo o código em módulos ou funções semanticamente coesos para mitigar limites de contexto.

• **2. Elicitação e Tradução Semântica via IA:** Submissão dos trechos de código e estruturas lógicas a LLMs calibrados por padrões formais de prompt (*Prompt Patterns*), como *Requirements Simulator* e *Cognitive Verifier*, solicitando ao modelo que extraia e descreva as regras de negócio subjacentes.

• **3. Formalização em Requisitos:** Transformação da linguagem técnica do código legado em artefatos compreensíveis por equipes de negócio, como *User Stories* e critérios de aceite estruturados em sintaxe natural/Gherkin (Dado-Quando-Então).

• **4. Revisão e Validação (*Human-in-the-Loop*):** Validação manual das regras extraídas por especialistas de domínio para corrigir inconsistências ou falsas premissas geradas pelo modelo.

B. Técnicas e Ferramentas de IA Utilizadas

• **Modelos de Linguagem (LLMs):** Utilização de modelos como GPT-4, Gemini Pro ou DeepSeek para interpretar código-fonte e inferir intenções de projeto em linguagem natural.

• **Engenharia de Prompt Especializada:** Aplicação de padrões conceituais catalogados, como o *Requirements Simulator* (para simular interações e descobrir lacunas), o *Cognitive Verifier* (para decompor regras complexas) e o *Fact Check List* (para validar premissas lógicas extraídas do código).

• **Integradores e RAG (Retrieval-Augmented Generation):** Conexão do código-fonte a bases vetoriais para manter o contexto do sistema completo durante a extração de requisitos.

C. Resultados Esperados e Limitações

• **Resultados Esperados:** Reconstrução rápida da especificação funcional e da arquitetura do sistema legado; redução do esforço manual de arqueologia de código; e criação de uma 'documentação viva' legível tanto por desenvolvedores quanto por partes interessadas.

• **Limitações:** Risco de **alucinações de IA** (geração de regras de negócio inexistentes); perda de contexto em bases de código extensas ou acopladas ('código espaguete'); e incapacidade de inferir intenções de negócio que não estejam explicitamente codificadas no software.

2. Reconstrução de Sistemas a partir dos Requisitos Extraídos com Inteligência Artificial

A. Descrição da Abordagem e Suas Etapas

A reconstrução utiliza a **Modernização Orientada a Comportamento e Síntese de Código via BDD (Behavior-Driven Development) e LLMs**. O sistema legado é reescrito em uma arquitetura moderna (ex.: microsserviços ou *Clean Architecture*) guiando a geração de código novo diretamente a partir dos requisitos formais extraídos.

• **1. Modelagem de Especificações Executáveis:** Conversão dos requisitos extraídos em cenários de teste BDD automatizáveis (utilizando Gherkin/Cucumber), estabelecendo o contrato de comportamento do novo sistema antes da codificação.

• **2. Geração Automatizada da Nova Arquitetura e Código:** Uso de LLMs para sintetizar a nova estrutura da aplicação (APIs REST, componentes de interface e serviços) em linguagens modernas (ex.: TypeScript, Java/Spring, Python) a partir das especificações e de padrões de design.

• **3. Síntese de Testes e Oráculos de Software:** Geração automatizada de suítes de teste funcionais e estruturais pela IA para cobrir os fluxos do novo sistema.

• **4. Execução e Validação Cruzada:** Comparação contínua do comportamento do novo sistema com o legado por meio de testes automatizados.

B. Técnicas e Ferramentas de IA Utilizadas

• **Ferramentas de Automação BDD/E2E:** *Cucumber*, *Cypress* e *Selenium* operando em conjunto com cenários Gherkin gerados ou refinados por IA.

• **Padrões de Prompt para Design e Refatoração:** Aplicação dos padrões *API Generator* (para transformar especificações funcionais em interfaces REST), *Code Clustering* e *Intermediate Abstraction* (para desacoplar dependências e evitar código monolítico).

• **Síntese de Código e Testes via LLM:** Utilização de comandos estruturados (*Persona* e *Template*) para que o modelo crie scripts executáveis com alta cobertura de instruções e ramificações.

C. Estratégias de Verificação e Preservação de Funcionalidades

• **Teste Diferencial Orientado à Diversidade (*AI-driven Differential Testing - AID*):** Execução paralela do sistema legado e do novo sistema utilizando as mesmas entradas (geradas por scripts de IA), identificando divergências nas saídas para detectar erros sutis de implementação.

• **Garantia de Cobertura de Código e Mutação:** Medição da abrangência dos testes automatizados via ferramentas como *JaCoCo* (cobertura de instruções e ramificações) e *PIT* (teste de mutação), garantindo que os testes gerados pela IA sejam capazes de detectar falhas reais no novo código.

• **Bateria de Testes de Regressão Automatizados:** Reexecução contínua dos cenários de aceite BDD para assegurar que nenhuma funcionalidade original seja degradada durante o processo de modernização.

3. Análise Crítica: Vantagens, Limitações e Desafios

Vantagens Principais

✓ **Aceleração Expressiva no Ciclo de Modernização:** Redução drástica do tempo exigido para mapear regras de negócio e reescrever aplicações, automatizando tarefas repetitivas.

✓ **Melhoria da Qualidade e Manutenibilidade do Código:** Substituição de código legado rígido por arquiteturas modulares, documentadas e acompanhadas por suítes robustas de testes automatizados.

✓ **Alinhamento entre TI e Negócio:** O uso de BDD e linguagem ubíqua (Gherkin) reduz ruídos de comunicação, permitindo que os requisitos extraídos pelo LLM sejam facilmente validados por analistas e clientes.

Limitações e Desafios

■ **Propagação de Débito Técnico e Regras Obsoletas:** Se o código legado contiver erros de lógica, regras de negócio desatualizadas ou gambiarras (*workarounds*), a IA pode interpretar esses desvios como comportamento correto e reproduzi-los no sistema reconstruído.

■ **Dependência da Qualidade do Prompt e Inconsistência (Não-determinismo):** LLMs podem apresentar variações e oscilações nas respostas para as mesmas instruções, exigindo rigor no uso de catálogos formais de prompts e supervisão humana constante.

■ **Casos Inválidos e Ajustes Manuais:** Como evidenciado em estudos experimentais de geração de testes via LLMs, é comum a ocorrência de casos de teste inválidos ou falhas de compilação que exigem intervenção humana manual, evidenciando que a IA atua como assistente e não como substituta completa do engenheiro de software.

4. Declaração do Uso de Inteligência Artificial

Declaração de Autoria e Transparência
Declaro que esta contribuição foi elaborada com o auxílio da ferramenta Gemini (NotebookLM), utilizada para apoiar a síntese teórica, a estruturação tópica das metodologias de engenharia de software e a organização das referências científicas conforme as diretrizes acadêmicas da disciplina.

5. Referências Bibliográficas

BELZNER, L.; GABOR, T.; WIRSING, M. **Large Language Model Assisted Software Engineering: prospects, challenges, and a case study preprint**. Cham: Springer, 2023. (Lecture Notes in Computer Science, v. 14380).

CAMPOS, A. G. A. **Proposta para implantação de automação de testes de software usando behavior driven development (BDD): Estudo de caso**. 2024. 67 f. Trabalho de Conclusão de Curso (Sistemas de Informação) – Universidade Federal de Uberlândia, Uberlândia, 2024.

LIU, K. et al. **LLM-powered test case generation for detecting tricky bugs**. In: ACM International Conference on Software Engineering (ICSE), 2024.

LIU, Z. et al. **Make LLM a testing expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions**. In: ICSE '24 – 46th International Conference on Software Engineering, Lisbon, Portugal, 2024.

OLIVEIRA, M. L. S. **Um estudo sobre a aplicação de LLMs no teste de software**. 2025. Monografia (Bacharelado em Tecnologia da Informação) – Universidade Federal Rural do Semi-Árido (UFERSA), Pau dos Ferros, 2025.

WHITE, J. et al. **ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design**. arXiv preprint arXiv:2303.07839, 2023.

WHITE, J. et al. **A prompt pattern catalog to enhance prompt engineering with ChatGPT**. arXiv preprint arXiv:2302.11382, 2023.

2. Anexos Práticos: Engenharia Reversa e BDD

Estes dois complementos práticos ilustram o ciclo de **extração de requisitos via prompt engineering** e sua conversão direta em **especificações BDD executáveis em sintaxe Gherkin** para a reconstrução do sistema.

Anexo Prático 1: Modelo de Prompt Estruturado para Extração de Requisitos

Este prompt utiliza a combinação dos padrões conceituais *Persona*, *Cognitive Verifier* e *Template* para instruir o LLM a analisar o código-fonte legado e extrair regras de negócio sem inventar comportamentos.

TEMPLATE DE PROMPT - ENGENHARIA REVERSA

```
PROMPT DE EXTRAÇÃO DE REQUISITOS (ENGENHARIA REVERSA)

[CONTEXTO E PERSONA]
Atue como um Engenheiro de Requisitos e Especialista em Engenharia Reversa de Software.
Sua tarefa é analisar o trecho de código-fonte legado fornecido a seguir e extrair as regras de negócio implícit
as, transformando a lógica técnica em requisitos funcionais claros.

[INSTRUÇÕES E REGRAS]
1. Aplique a verificação cognitiva: analise o fluxo de controle, condicionais e variáveis antes de formular a re
sposta.
2. Identifique cada regra de negócio (RN) distinta codificada na rotina.
3. Se houver ambiguidades ou lacunas no código, declare-as explicitamente como "Premissas/Avisos". Não invente r
egras de negócio não presentes no código.

[FORMATO DE SAÍDA - TEMPLATE]
Gere a resposta estritamente na seguinte estrutura:
- Visão Geral do Módulo: (Breve resumo do propósito funcional do código)
- Regras de Negócio Extraídas:
  * [RN01]: Nome da Regra - Descrição detalhada da condição e do resultado.
  * [RN02]: Nome da Regra - Descrição detalhada da condição e do resultado.
- Entradas e Saídas Mapeadas: (Tabela/lista com parâmetros e tipos)
- Critérios de Aceite Sugeridos (Gherkin):
  Dado [contexto inicial]
  Quando [ação/evento]
  Então [resultado esperado]

[CÓDIGO-FONTE LEGADO PARA ANÁLISE]
<cole_aqui_o_codigo_fonte_legado_ex_cobol_ou_java_legado>
```

Anexo Prático 2: Especificação de Cenários BDD em Sintaxe Gherkin

Após a extração dos requisitos pelo LLM, a especificação é formalizada na linguagem ubíqua Gherkin. Isso estabelece a documentação viva e o contrato executável de comportamento que guiará a reescrita do sistema e a automação de testes de regressão (ex.: Cucumber/Cypress).

ESPECIFICAÇÃO BDD EXECUTÁVEL (GHERKIN)

```
#language: pt
Funcionalidade: Processamento e Atualização de Precificação de Produtos
  Como Analista de Negócios e Sistema de Precificação
  Eu quero que o novo sistema processe os cálculos de margem e descontos automaticamente
  Para garantir a precisão das taxas de venda sem regressões em relação ao sistema legado

  Contexto:
    Dado que a base de dados de produtos está integrada e sincronizada
    E o usuário possui permissão de "Analista de Precificação"

  Cenário: Aplicação bem-sucedida de desconto por volume
    Dado que existe um produto cadastrado com preço base de "R$ 100,00"
    Quando o usuário solicitar a precificação para uma quantidade de "50" unidades
    Então o sistema deve aplicar o percentual de desconto de "10%"
    E o valor final unitário calculado deve ser "R$ 90,00"
    E o status do cálculo deve ser registrado como "Aprovado"

  Cenário: Validação de limite máximo de desconto permitido
    Dado que o limite máximo de desconto configurado na regra de negócio é "25%"
    Quando o usuário tentar aplicar manualmente um desconto de "30%"
    Então o sistema deve rejeitar a operação
    E deve exibir a mensagem de erro "Desconto acima do limite permitido para a categoria"
    E o preço original do produto não deve ser alterado
```

3. Qualidade de Testes de Software com IA: JaCoCo e PIT

No contexto da modernização e reconstrução de software orientada por IA, as ferramentas **JaCoCo** e **PIT** atuam de forma complementar para responder a duas perguntas cruciais sobre o código reconstruído:

As Duas Perguntas Fundamentais da Qualidade de Testes
1. **O JaCoCo responde:** *'Os testes estão exercitando todas as partes e caminhos lógicos do novo código?'* (Abrangência Estrutural).

2. **O PIT responde:** *'Esses testes são realmente capazes de detectar erros ou só estão rodando o código sem validar nada?'* (Eficácia na Detecção de Falhas).

1. O que o JaCoCo faz? (Java Code Coverage)

O **JaCoCo** é uma biblioteca de análise de cobertura de código para Java. Durante a execução dos testes gerados pela IA, ele monitora diretamente a execução do *bytecode* e mede a extensão do código que foi exercitada.

Métricas Fundamentais do JaCoCo:

• **Cobertura de Instruções (*Instruction Coverage*):** Analisa a menor unidade do código (instruções individuais de *bytecode* Java), indicando a porcentagem exata de instruções que foram efetivamente executadas.

• **Cobertura de Ramificações (*Branch Coverage*):** Avalia todas as decisões lógicas condicionais do programa (como estruturas if e switch). Ele verifica se o teste percorreu **tanto o caminho verdadeiro quanto o falso** de cada condicional, garantindo que nenhum fluxo de controle fique sem validação.

Por que ele é essencial na modernização com IA? Para garantir que o novo sistema desenvolvido a partir das especificações BDD/Gherkin não deixe linhas ou caminhos lógicos 'cegos' ou esquecidos durante a reescrita.

2. O que o PIT faz? (PITest / Parallel Isolated Test)

O **PIT** é uma ferramenta voltada para **Teste de Mutação** (*Mutation Testing*). Ele avalia a **qualidade intrínseca da suíte de testes**, e não apenas o código da aplicação.

• **1. Criação de 'Mutantes':** O PIT introduz pequenas modificações deliberadas no código-fonte compilado para simular defeitos reais (por exemplo, altera > para <, inverte uma condição booleana ou remove uma chamada de método).

• **2. Execução dos Testes:** O PIT roda a suíte de testes contra cada versão modificada (o código mutante).

• **3. Morte do Mutante:** Se algum teste falhar diante dessa alteração, o mutante é considerado **'morto'** (o que é o resultado desejado, pois prova que o teste percebeu a falha). Se todos os testes passarem mesmo com o erro inserido, o mutante 'sobrevive', revelando que o teste é fraco.

• **4. Escore de Mutação (*Mutation Score*):** É a porcentagem de mutantes eliminados em relação ao total gerado. Quanto mais próximo de 100%, maior é a capacidade dos testes em identificar defeitos reais.

3. A Complementaridade entre JaCoCo e PIT na nossa Proposta

Uma das grandes armadilhas no uso de IA para sintetizar código ou testes é a **falsa sensação de qualidade**:

■ Uma IA pode facilmente gerar testes que atingem **100% de cobertura no JaCoCo** apenas chamando os métodos, mas com asserções fracas ou ausentes (sem verificar o valor de retorno).

■ Se colocarmos apenas o JaCoCo, parece que o sistema está 100% validado; contudo, se inserirmos um bug, o teste continuará passando.

■ **O PIT entra justamente para resolver esse problema:** ao injetar bugs artificiais, ele comprova se as asserções criadas pela IA realmente acusam o erro.

Síntese Metodológica
Em resumo: O **JaCoCo afere a abrangência** (quantidade de código percorrida) e o **PIT afere a efetividade** (força e capacidade real dos testes em barrar falhas reais e mutações no código).